

# JS Timing Functions

## 1. What are Timing Functions in JS?

- a. *Timing functions* in JavaScript allow us to execute a block of code after a certain *delay* or *repeatedly* after specific time intervals.
- b. They are essential for creating delays, repeated tasks, or handling time based actions in web applications.

## 2. Why do we need timing functions in JS?

- a. *Scheduling tasks*: Run code after a specific time delay (e.g., showing a popup after 3 seconds).
- b. *Repeating tasks*: Run code repeatedly (e.g., updating a clock every second).

## 3. Types of Timing Functions:

- a. JavaScript provides four main timing functions:

- i. `setTimeout()`
- ii. `setInterval()`
- iii. `clearTimeout()`
- iv. `clearInterval()`

## 4. `setTimeout()`:

- a. The *`setTimeout()`* executes the function once after a specified delay (in milliseconds).
- b. The `setTimeout()` is executed only once.
- c. **Syntax** →



```
let timerId = setTimeout(callback, timeInMilliseconds);
```

d. **For Example** →

```
setTimeout(() => {  
    console.log("Hello World!");  
}, 2000); // This runs after 2 seconds
```

## 5. setInterval():

a. The **setInterval()** executes the function repeatedly at specified time intervals (in milliseconds).

b. **Syntax** →

```
 let intervalId = setInterval(callback, timeInMilliseconds);
```

c. Perform tasks repeatedly (e.g., updating a countdown or clock).

d. **For Example** →

```
let count = 0;  
setInterval(() => {  
    console.log(`Counter: ${++count}`);  
}, 1000);
```

## 6. clearTimeout():

a. The **clearTimeout()** stops a timeout set by **setTimeout()**.

b. **Syntax** →

```
 clearTimeout(timerId);
```

c. Cancel a delayed task if it's no longer needed.

d. **For Example** →

```
let timerId = setTimeout(() => {  
    console.log("This will not execute");  
}, 2000);
```

```
// Cancel the timeout before it runs  
clearTimeout(timerId);
```

## 7. clearInterval():

a. The `clearInterval()` stops a repeating task set by `setInterval()`.

b. **Syntax** →

```
 clearInterval(intervalId);
```

c. Stop a repeated action when a condition is met.

d. **For Example** →

```
let count = 0;  
let myInterval = setInterval(() => {  
  console.log(`Counter: ${++count}`);  
  if (count == 5) {  
    clearInterval(myInterval);  
  }  
, 1000);
```

## 8. Key Points to Remember:

a. **Milliseconds:** Time in these functions is specified in (milliseconds 1 second = 1000 ms).

b. **setTimeout vs setInterval :** setTimeout Executes a task once after the delay. setInterval Executes a task repeatedly at fixed intervals.

c. **Clearing functions:**

i. `clearTimeout()`: stops a delayed task.

ii. `clearInterval()`: stops a repeating task.

d. **IDs returned:** Both setTimeout and setInterval return an ID. You need this ID to clear the timeout/interval.